DATABASE MANAGEMENT SYSTEMS (CSA0501)

Assignment 1 – CO3

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.

Assignment Title: Analyse, Normalise and Validate the Relational Schema of an Online Retail Order

Name: SK ALI BABU

Register No.: 192524038

Submission date: 31/10/2026

Total marks: 50

Nature: Individual

Duration: 1 week

Important assumptions and scope

The answers follow the ShopNest scenario in the supplied assignment. The scenario does not provide actual sample rows or measured benchmark results, so a consistent sample instance is constructed below and a reproducible experiment plan is supplied. No benchmark values are presented as if they were measured. The result tables in Tasks 9–10 are templates to fill after running the scripts on the student's computer.

- One order contains at most one line for a given ProductID. Therefore (OrderID, ProductID) is the candidate key. If the business permits the same product on multiple separate lines in one order, add OrderLineNo and use (OrderID, OrderLineNo) instead.
- The stated current UnitPrice is functionally determined by ProductID in the original scenario. The historical price actually paid is handled separately in Task 12.
- The MVDs for tags and colours are independent: every tag can occur with every available colour for the product.
- For address dependencies, the scenario states PIN → city and state and street + city → PIN. These rules are retained as written.

TASK 1 – Identify and Formalise the Functional Dependencies (5 Marks)

1.1 Functional dependencies

A functional dependency X → Y means that for any two rows, equal values of X require equal values of Y. The following non-trivial dependencies follow from the stated business rules:

Dependency	Business-rule justification
CustomerID → CustomerName, CustomerEmail	A customer identifier identifies one customer record.
OrderID → OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN	Each order belongs to one customer, has one date and is shipped to one address.
ShipPIN → ShipCity, ShipState	A PIN code belongs to one city and state.
(ShipStreet, ShipCity) → ShipPIN	A street in a city has one PIN code.
ProductID → ProductName, UnitPrice, CategoryID, SupplierID	A product has one current price, category and supplier.
CategoryID → CategoryName	A category identifier identifies its name.
SupplierID → SupplierName, SupplierPhone	A supplier identifier identifies supplier details.
WarehouseID → WarehouseCity	A warehouse identifier identifies its city.
(OrderID, ProductID) → Quantity, WarehouseID	A particular product line in an order has one quantity and dispatch warehouse.
ProductID → CategoryName	Derived transitively through ProductID → CategoryID → CategoryName.
ProductID → SupplierName, SupplierPhone	Derived transitively through ProductID → SupplierID → supplier details.

The last two are useful transitive consequences, but they are not counted as independent base dependencies when constructing the minimal cover.

1.2 Multivalued dependencies

ProductID ↠ Tag and ProductID ↠ Colour hold because a product may have several search tags and several colours, and the two sets are independent. Knowing a product and one tag does not determine a single colour.

1.3 Sample instance (12 tuples)

The following compact sample uses one row per order-product line. Repeated customer, product, supplier, category and warehouse facts are intentional and demonstrate the redundancy in the original flat table. TagList and ColourList are shown as sets, so the displayed table is not yet in 1NF.

Order	Date	Cust	Name	Email	Street	City	PIN	State	Prod	Product name	Price	Cat	Category	Supp	Supplier	Phone	Qty	Wh	Whcity	Tags	Colours
O101	01-10-2026	C01	Asha	asha@mail.test	12 Lake Rd	Kakinada	533001	AP	P01	Pen	10	CAT1	Stationery	S01	Nova	900001	2	W1	Kakinada	writing;office	Blue;Black
O101	01-10-2026	C01	Asha	asha@mail.test	12 Lake Rd	Kakinada	533001	AP	P02	Notebook	40	CAT1	Stationery	S01	Nova	900001	1	W1	Kakinada	paper;office	Red;Blue
O102	02-10-2026	C02	Bala	bala@mail.test	8 Park St	Rajahmundry	533101	AP	P01	Pen	10	CAT1	Stationery	S01	Nova	900001	5	W2	Rajahmundry	writing;office	Blue;Black
O103	03-10-2026	C03	Charan	charan@mail.test	4 Main Rd	Vijayawada	520001	AP	P03	Bottle	120	CAT2	Accessories	S02	Orbit	900002	1	W1	Kakinada	water;travel	Green;Blue
O104	04-10-2026	C01	Asha	asha@mail.test	12 Lake Rd	Kakinada	533001	AP	P02	Notebook	40	CAT1	Stationery	S01	Nova	900001	3	W2	Rajahmundry	paper;office	Red;Blue
O105	05-10-2026	C04	Divya	divya@mail.test	6 Hill St	Kakinada	533002	AP	P04	Bag	650	CAT2	Accessories	S02	Orbit	900002	1	W1	Kakinada	school;travel	Black;Green
O106	06-10-2026	C02	Bala	bala@mail.test	8 Park St	Rajahmundry	533101	AP	P03	Bottle	120	CAT2	Accessories	S02	Orbit	900002	2	W2	Rajahmundry	water;travel	Green;Blue
O107	07-10-2026	C05	Esha	esha@mail.test	2 Beach Rd	Kakinada	533003	AP	P05	Marker	25	CAT1	Stationery	S01	Nova	900001	4	W1	Kakinada	writing;art	Red;Black
O1	08	C0	Cha	charan@	4	Vijaya	520	AP	P0	Pen	10	CA	Station	S0	Nova	900	2	W	Rajahm	writing;	Blue;Bl

08	-10-2026	3	ran	mail.test	Main Rd	wada	001		1			T1	ery	1		001		2	undry	office	ack
O109	09-10-2026	C04	Divya	divya@mail.test	6 Hill St	Kakinada	533002	AP	P04	Bag	650	CAT2	Accessories	S02	Orbit	900002	2	W2	Rajahmundry	school;travel	Black;Green
O110	10-10-2026	C05	Esha	esha@mail.test	2 Beach Rd	Kakinada	533003	AP	P02	Notebook	40	CAT1	Stationery	S01	Nova	900001	2	W1	Kakinada	paper;office	Red;Blue
O111	11-10-2026	C02	Bala	bala@mail.test	8 Park St	Rajahmundry	533101	AP	P05	Marker	25	CAT1	Stationery	S01	Nova	900001	1	W2	Rajahmundry	writing;art	Red;Black

1.4 Dependency diagram

Candidate key: (OrderID, ProductID)\n(OrderID, ProductID) ⟶FULL⟶ Quantity, WarehouseID\nOrderID ⟶PARTIAL⟶ OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN\nCustomerID ⟶TRANSITIVE⟶ CustomerName, CustomerEmail\nProductID ⟶PARTIAL⟶ ProductName, UnitPrice, CategoryID, SupplierID\nCategoryID ⟶TRANSITIVE⟶ CategoryName\nSupplierID ⟶TRANSITIVE⟶ SupplierName, SupplierPhone\nWarehouseID ⟶TRANSITIVE⟶ WarehouseCity\nProductID ↠ Tag; ProductID ↠ Colour

Full dependency: Quantity and WarehouseID require the complete line key. Partial dependency: order facts depend only on OrderID and product facts depend only on ProductID. Transitive dependency: descriptive facts depend on a non-key determinant reached through another dependency.

TASK 2 – Identify the Redundancy and Anomalies (2 Marks)

The predominant problem is repetition of the same descriptive fact across many order lines (tuple-level redundancy), not simply a large number of rows. Ten thousand rows containing ten thousand different facts are not redundant; the same supplier phone repeated in thousands of rows is redundant.

- Insertion anomaly: a new supplier S03 cannot be stored in ORDER_LINE until at least one order line for one of its products exists. For example, supplier S03 with phone 900003 has no independent row in which it can be inserted.
- Deletion anomaly: deleting the only line for product P05 in order O111 could also remove the last stored copy of P05’s product details and supplier/category facts if no other order refers to them.

- Update anomaly: changing supplier S01 phone from 900001 to 900009 requires updating every row containing S01. If one row is missed, the table contains conflicting phone numbers.

As order lines grow by about 20,000 per day, customer, product, category and supplier facts are copied into each new line. Those master facts grow slowly, while their repeated copies grow roughly in proportion to order-line count.

TASK 3 – Compute Closures, Keys and the Canonical Cover (6 Marks)

3.1 Attribute closures

Closure	Step-by-step result
$OrderID^+$	Start {OrderID}; OrderID → OrderDate, CustomerID, address fields; CustomerID gives name/email; PIN gives city/state. Result: {OrderID, OrderDate, CustomerID, CustomerName, CustomerEmail, ShipStreet, ShipCity, ShipPIN, ShipState}.
$ProductID^+$	ProductID gives ProductName, UnitPrice, CategoryID, SupplierID; CategoryID gives CategoryName; SupplierID gives SupplierName and SupplierPhone. It does not give OrderID, Quantity or WarehouseID.
$(OrderID, ProductID)^+$	Apply OrderID and ProductID dependencies above; the pair also gives Quantity and WarehouseID; WarehouseID gives WarehouseCity. This produces all single-valued ORDER_LINE attributes.
$(OrderID, Quantity)^+$	OrderID gives order/customer/address attributes, but Quantity does not identify ProductID or WarehouseID. Not a key.
$(ProductID, WarehouseID)^+$	Gives product/category/supplier facts and warehouse city, but not OrderID or Quantity

3.2 Candidate key and minimality proof

Under the stated one-line-per-product-per-order assumption, the only candidate key is (OrderID, ProductID). Its closure contains every single-valued attribute, so it is a superkey. Neither

component can be removed: OrderID$^+$ lacks product and line facts; ProductID$^+$ lacks order and line facts. Hence it is minimal.

No other candidate key exists because neither OrderID nor ProductID can be derived from any dependency whose right-hand side contains it. Every key must therefore contain both attributes. The minimal such set is the pair.

Prime attributes: OrderID and ProductID. Non-prime attributes: all remaining attributes in the flat relation. Tag and Colour are repeating multivalued facts, not single-valued columns in the normalized design.

3.3 Canonical cover

Split right-hand sides into single attributes, remove duplicate/transitively implied dependencies, and retain the following minimal determinant rules (grouped again for readability):

```
OrderID → OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN\nCustomerID → CustomerName, CustomerEmail\nShipPIN → ShipCity, ShipState\n(ShipStreet, ShipCity) → ShipPIN\nProductID → ProductName, UnitPrice, CategoryID, SupplierID\nCategoryID → CategoryName\nSupplierID → SupplierName, SupplierPhone\nWarehouseID → WarehouseCity\n(OrderID, ProductID) → Quantity, WarehouseID
```

Extraneous-attribute checks: removing OrderID from the composite determinant does not derive Quantity; removing ProductID does not derive Quantity. Removing either ShipStreet or ShipCity from (ShipStreet, ShipCity) → ShipPIN does not follow from the remaining dependencies. Each single determinant in the other dependencies is necessary because its right-side fact cannot be derived without it. Dependencies such as ProductID → CategoryName are omitted from the cover because they follow transitively from ProductID → CategoryID and CategoryID → CategoryName.

Equivalence: every dependency in the cover is in F or is obtained by splitting a right-hand side (decomposition). Every omitted transitive dependency is obtained by transitivity; for example ProductID → CategoryID and CategoryID → CategoryName imply ProductID → CategoryName. Thus $F^+ = F\square^+$ for the stated base dependency set, by Armstrong's reflexivity, augmentation and transitivity rules.

TASK 4 – Analyse First and Second Normal Form (4 Marks)

4.1 First Normal Form

1NF requires each cell to contain one atomic value and each row to be uniquely identifiable. TagList and ColourList contain repeating sets, so the flat representation violates 1NF.

- Option 1: store one tag/colour per row. This makes values atomic but creates a tag × colour product and repeats all order/product fields.
- Option 2: move repeating values to separate child tables PRODUCT_TAG(ProductID, Tag) and PRODUCT_COLOUR(ProductID, Colour). This avoids cross-product combinations and keeps the independent sets separate; it requires joins to retrieve them.

4.2 Second Normal Form

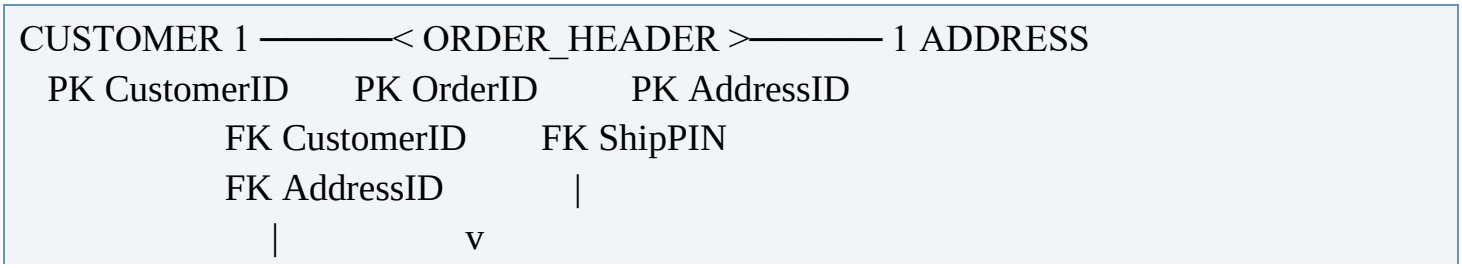
2NF requires 1NF and no non-prime attribute to depend on a proper subset of a candidate key. With key (OrderID, ProductID), order date/customer/address attributes depend only on OrderID; product name/price/category/supplier attributes depend only on ProductID. These are partial dependencies.

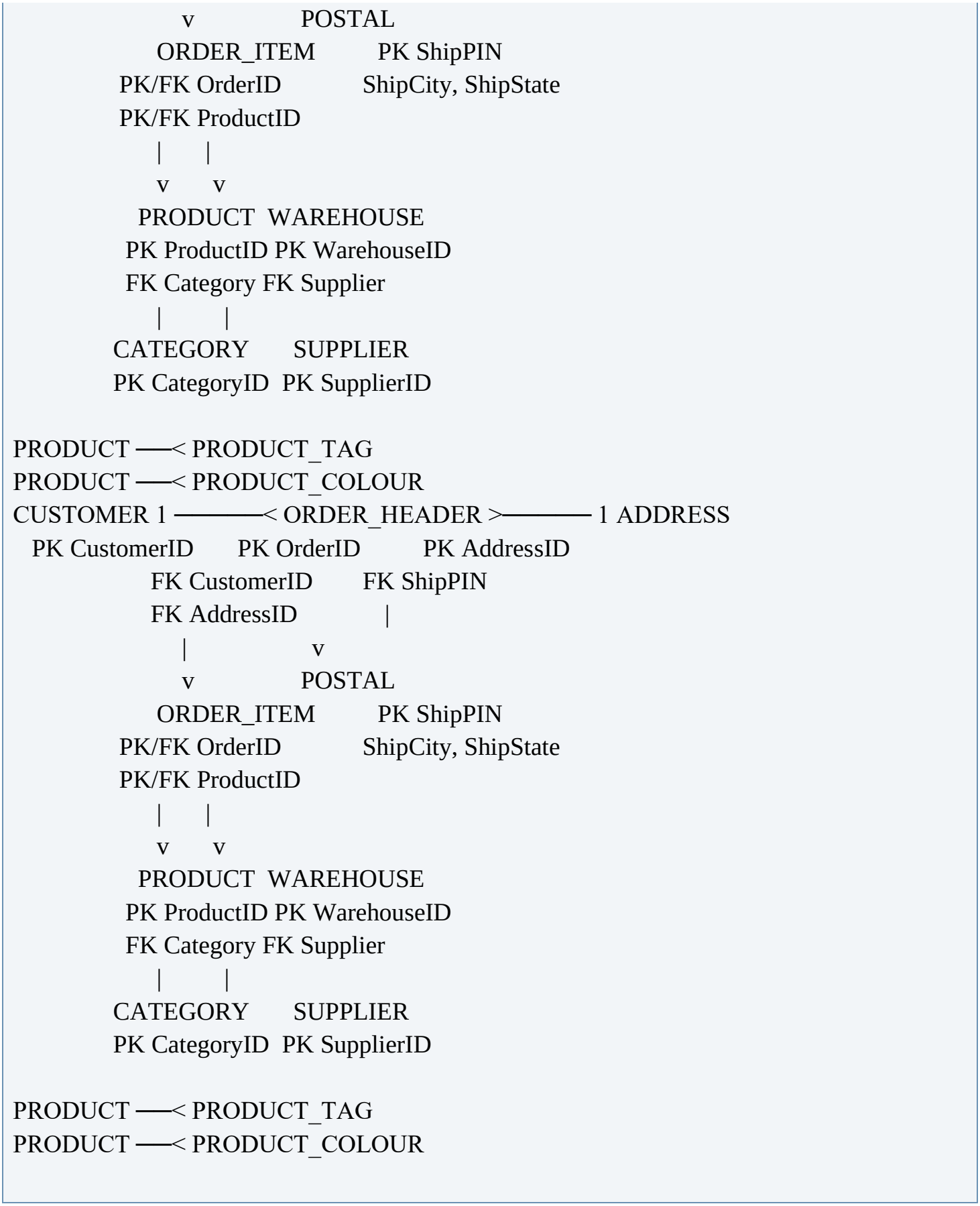
Relation after 2NF split	Primary key	Main dependencies preserved
ORDER_HEADER(OrderID, OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN)	OrderID	OrderID → order date, customer and shipping address
PRODUCT(ProductID, ProductName, UnitPrice, CategoryID, SupplierID)	ProductID	ProductID → product name, current price, category and supplier
ORDER_ITEM(OrderID, ProductID, Quantity, WarehouseID)	(OrderID, ProductID)	(OrderID, ProductID) → Quantity, WarehouseID
CUSTOMER(CustomerID, CustomerName, CustomerEmail)	CustomerID	CustomerID → customer name, email
CATEGORY(CategoryID, CategoryName)	CategoryID	CategoryID → CategoryName
SUPPLIER(SupplierID, SupplierName, SupplierPhone)	SupplierID	SupplierID → supplier name, phone
WAREHOUSE(WarehouseID, WarehouseCity)	WarehouseID	WarehouseID → WarehouseCity

The split ensures that changing a customer, product or supplier fact requires one master-row update. It also allows master data to be inserted without an order and prevents deletion of an order from deleting the master entity.

TASK 5 – Design the 3NF and BCNF Decompositions (3 Marks)

Relational schema relationship diagram





5.1 3NF decomposition

The 2NF relations still contain transitive dependencies. ProductID → SupplierID → SupplierName/SupplierPhone and ProductID → CategoryID → CategoryName are removed by

keeping supplier and category details in their own relations. Likewise, WarehouseID → WarehouseCity is kept in WAREHOUSE. Address dependencies are separated into ADDRESS/PIN relations.

3NF relation	Key	Functional dependency
CUSTOMER(CustomerID, CustomerName, CustomerEmail)	CustomerID	CustomerID → name, email
ORDER_HEADER(OrderID, OrderDate, CustomerID, AddressID)	OrderID	OrderID → date, customer, address
ADDRESS(AddressID, ShipStreet, ShipCity, ShipPIN)	AddressID	AddressID → street, city, PIN
POSTAL(ShipPIN, ShipCity, ShipState)	ShipPIN	ShipPIN → city, state
PRODUCT(ProductID, ProductName, UnitPrice, CategoryID, SupplierID)	ProductID	ProductID → product attributes
CATEGORY(CategoryID, CategoryName)	CategoryID	CategoryID → CategoryName
SUPPLIER(SupplierID, SupplierName, SupplierPhone)	SupplierID	SupplierID → supplier details
WAREHOUSE(WarehouseID, WarehouseCity)	WarehouseID	WarehouseID → WarehouseCity
ORDER_ITEM(OrderID, ProductID, Quantity, WarehouseID)	(OrderID, ProductID)	line key → quantity, warehouse

The scenario also gives (ShipStreet, ShipCity) → ShipPIN. Enforce it with a UNIQUE constraint on (ShipStreet, ShipCity) in ADDRESS. The postal table separately enforces ShipPIN → ShipCity, ShipState.

5.2 BCNF check

A relation is in BCNF when every non-trivial FD determinant is a superkey. The relations above are BCNF under the stated dependencies, provided the address rule is represented carefully. In ADDRESS, AddressID is the key and (ShipStreet, ShipCity) is also a candidate key because it determines ShipPIN and AddressID identifies the address. Add UNIQUE(ShipStreet, ShipCity). In POSTAL, ShipPIN is the key. All remaining determinants are keys.

5.3 Lossless join and dependency preservation

Each decomposition separates attributes using a determinant that is a key in one resulting relation. For a binary split R into R1 and R2, the join is lossless when (R1 ∩ R2) → R1 or (R1 ∩ R2) → R2. For example, ORDER_HEADER and CUSTOMER intersect on CustomerID, which is the key of CUSTOMER; joining them cannot invent a different customer. ORDER_ITEM joins to ORDER_HEADER on OrderID and to PRODUCT on ProductID; the referenced identifiers are primary/foreign keys.

The FDs are preserved locally: customer, product, category, supplier, warehouse, postal and order-item dependencies are checked in their own tables. The composite address rule is enforced by UNIQUE(ShipStreet, ShipCity). Cross-table rules not represented by a single relation may require a trigger or application transaction.

TASK 6 – Analyse Multivalued Dependencies and 4NF (3 Marks)

An MVD X ↠ Y holds when, for a fixed X, the set of Y values is independent of the remaining attributes. For each product, tags and colours vary independently; therefore ProductID ↠ Tag and ProductID ↠ Colour.

6.1 Tuple-swapping verification

ProductID	Tag	Colour
P01	writing	Blue
P01	writing	Black
P01	office	Blue
P01	office	Black
P02	paper	Red
P02	paper	Blue
P02	office	Red
P02	office	Blue

For P01, the tuples (P01, writing, Blue) and (P01, office, Black) require the swapped tuples (P01, writing, Black) and (P01, office, Blue), both present. The same holds for P02. This demonstrates the independent combination rule.

6.2 4NF decomposition and row count

The BCNF relation PRODUCT_TAG_COLOUR(ProductID, Tag, Colour) may be in BCNF because ProductID does not functionally determine one Tag or Colour. However, it stores every combination. With t tags and c colours it contains t × c rows for one product. For 3 tags and 4 colours, it stores 12 rows.

Decompose into PRODUCT_TAG(ProductID, Tag) and PRODUCT_COLOUR(ProductID, Colour). The same product now needs t + c rows: 3 + 4 = 7 rows, instead of 12. The decomposition removes the non-trivial MVDs from each relation and is in 4NF. Every 4NF

relation is in BCNF because an FD is a special case of an MVD; the reverse is false because BCNF does not eliminate all non-trivial MVDs.

TASK 7 – Analyse Join Dependencies and 5NF (4 Marks)

Let SUPPLY(SupplierID, CategoryID, WarehouseID) represent a valid supplier-category-warehouse triple. The stated cyclic business rule says a triple is valid when the supplier supplies the category, the category is stocked at the warehouse, and the supplier delivers to that warehouse.

7.1 Join dependency

The join dependency *((SupplierID, CategoryID), (CategoryID, WarehouseID), (SupplierID, WarehouseID)) states that SUPPLY equals the natural join of its three binary projections: SC, CW and SW. It is a genuine ternary constraint; none of the three pairwise facts alone determines the third attribute.

SupplierID	CategoryID	WarehouseID
S1	C1	W1
S1	C2	W2
S2	C1	W2
S2	C2	W1
S3	C1	W1
S3	C2	W2

Projecting the six tuples gives SC={(S1,C1),(S1,C2),(S2,C1),(S2,C2),(S3,C1),(S3,C2)}, CW={(C1,W1),(C2,W2),(C1,W2),(C2,W1)}, and SW={(S1,W1),(S1,W2),(S2,W2),(S2,W1),(S3,W1),(S3,W2)}. Joining these projections reconstructs the listed valid triples only if the cyclic rule is a true business constraint and the instance satisfies it. The chase checks whether the tuple symbols become identical in the joined tableau; in this case, the JD is the required constraint, not something inferable from ordinary FDs.

A two-projection decomposition is generally lossy here: for example, projecting on SC and CW can combine a supplier-category pair with a category-warehouse pair without checking that the supplier delivers to that warehouse. The missing SW relationship permits spurious triples. The same issue occurs if any one of the three pairwise projections is omitted.

7.2 5NF decomposition

Binary relation	Primary key
SUPPLIER_CATEGORY(SupplierID, CategoryID)	(SupplierID, CategoryID)
CATEGORY_WAREHOUSE(CategoryID, WarehouseID)	(CategoryID, WarehouseID)

SUPPLIER_WAREHOUSE(SupplierID, WarehouseID)	(SupplierID, WarehouseID)

For the six-tuple sample, the original SUPPLY has 6 rows. The projections contain 6, 4 and 6 rows respectively, totalling 16 stored pair rows before considering index overhead. Joining all three projections under the valid JD reconstructs 6 triples. 5NF (project-join normal form) requires every non-trivial join dependency to be implied by candidate keys. It is rarely enforced because extra tables and joins increase implementation and query complexity. It is useful when a genuine cyclic rule makes the ternary relation exactly reconstructible from all three pairwise facts.

Normalization pathway diagram

```
ORDER_LINE (UNNORMALIZED)
      |
      | split repeating TagList / ColourList
      v
1NF: atomic values
      |
      | remove partial dependencies on OrderID / ProductID
      v
2NF: ORDER_HEADER + PRODUCT + ORDER_ITEM + master tables
      |
      | remove transitive dependencies
      v
3NF / BCNF: CUSTOMER, PRODUCT, SUPPLIER, CATEGORY,
          WAREHOUSE, POSTAL, ADDRESS, ORDER_ITEM
      |
      | separate independent multivalued facts
      v
4NF: PRODUCT_TAG + PRODUCT_COLOUR
      |
      | apply valid ternary join dependency
      v
5NF: SUPPLIER_CATEGORY + CATEGORY_WAREHOUSE
    + SUPPLIER_WAREHOUSE
ORDER_LINE (UNNORMALIZED)
      |
      | split repeating TagList / ColourList
      v
```

```
1NF: atomic values
      |
      | remove partial dependencies on OrderID / ProductID
      v
2NF: ORDER_HEADER + PRODUCT + ORDER_ITEM + master tables
      |
      | remove transitive dependencies
      v
3NF / BCNF: CUSTOMER, PRODUCT, SUPPLIER, CATEGORY,
          WAREHOUSE, POSTAL, ADDRESS, ORDER_ITEM
      |
      | separate independent multivalued facts
      v
4NF: PRODUCT_TAG + PRODUCT_COLOUR
      |
      | apply valid ternary join dependency
      v
5NF: SUPPLIER_CATEGORY + CATEGORY_WAREHOUSE
    + SUPPLIER_WAREHOUSE
```

TASK 8 – Implement and Test the Schemas (4 Marks)

The following MySQL 8+ DDL provides a normalized implementation. It uses primary keys, foreign keys, UNIQUE, NOT NULL and CHECK constraints. The unnormalized comparison table can be created separately for anomaly demonstrations.

8.1 Normalized MySQL schema

```
CREATE DATABASE IF NOT EXISTS shopnest;
USE shopnest;

CREATE TABLE customer (
  customer_id VARCHAR(10) PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  customer_email VARCHAR(150) NOT NULL UNIQUE
);
CREATE TABLE supplier (
  supplier_id VARCHAR(10) PRIMARY KEY,
  supplier_name VARCHAR(100) NOT NULL,
  supplier_phone VARCHAR(20) NOT NULL
);
```

```
CREATE TABLE category (
  category_id VARCHAR(10) PRIMARY KEY,
  category_name VARCHAR(80) NOT NULL UNIQUE
);
CREATE TABLE warehouse (
  warehouse_id VARCHAR(10) PRIMARY KEY,
  warehouse_city VARCHAR(80) NOT NULL
);
CREATE TABLE postal (
  ship_pin VARCHAR(10) PRIMARY KEY,
  ship_city VARCHAR(80) NOT NULL,
  ship_state VARCHAR(80) NOT NULL
);
CREATE TABLE address (
  address_id BIGINT PRIMARY KEY,
  ship_street VARCHAR(150) NOT NULL,
  ship_city VARCHAR(80) NOT NULL,
  ship_pin VARCHAR(10) NOT NULL,
  UNIQUE (ship_street, ship_city),
  FOREIGN KEY (ship_pin) REFERENCES postal(ship_pin)
);
CREATE TABLE product (
  product_id VARCHAR(10) PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
  category_id VARCHAR(10) NOT NULL,
  supplier_id VARCHAR(10) NOT NULL,
  FOREIGN KEY (category_id) REFERENCES category(category_id),
  FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id)
);
CREATE TABLE order_header (
  order_id VARCHAR(12) PRIMARY KEY,
  order_date DATE NOT NULL,
  customer_id VARCHAR(10) NOT NULL,
  address_id BIGINT NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
  FOREIGN KEY (address_id) REFERENCES address(address_id)
);
CREATE TABLE order_item (
```

```
  order_id VARCHAR(12) NOT NULL,
  product_id VARCHAR(10) NOT NULL,
  quantity INT NOT NULL CHECK (quantity > 0),
  warehouse_id VARCHAR(10) NOT NULL,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id) REFERENCES order_header(order_id),
  FOREIGN KEY (product_id) REFERENCES product(product_id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouse(warehouse_id)
);
CREATE TABLE product_tag (
  product_id VARCHAR(10) NOT NULL,
  tag VARCHAR(60) NOT NULL,
  PRIMARY KEY (product_id, tag),
  FOREIGN KEY (product_id) REFERENCES product(product_id)
);
CREATE TABLE product_colour (
  product_id VARCHAR(10) NOT NULL,
  colour VARCHAR(40) NOT NULL,
  PRIMARY KEY (product_id, colour),
  FOREIGN KEY (product_id) REFERENCES product(product_id)
);
CREATE TABLE supplier_category (
  supplier_id VARCHAR(10) NOT NULL,
  category_id VARCHAR(10) NOT NULL,
  PRIMARY KEY (supplier_id, category_id),
  FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id),
  FOREIGN KEY (category_id) REFERENCES category(category_id)
);
CREATE TABLE category_warehouse (
  category_id VARCHAR(10) NOT NULL,
  warehouse_id VARCHAR(10) NOT NULL,
  PRIMARY KEY (category_id, warehouse_id),
  FOREIGN KEY (category_id) REFERENCES category(category_id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouse(warehouse_id)
);
CREATE TABLE supplier_warehouse (
  supplier_id VARCHAR(10) NOT NULL,
  warehouse_id VARCHAR(10) NOT NULL,
  PRIMARY KEY (supplier_id, warehouse_id),
```

```
  FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouse(warehouse_id)
);
```

8.2 Unnormalized comparison table

```
CREATE TABLE order_line_flat (
  order_id VARCHAR(12), order_date DATE, customer_id VARCHAR(10),
  customer_name VARCHAR(100), customer_email VARCHAR(150),
  ship_street VARCHAR(150), ship_city VARCHAR(80), ship_pin VARCHAR(10),
  ship_state VARCHAR(80), product_id VARCHAR(10), product_name VARCHAR(100),
  unit_price DECIMAL(10,2), category_id VARCHAR(10), category_name VARCHAR(80),
  supplier_id VARCHAR(10), supplier_name VARCHAR(100), supplier_phone
VARCHAR(20),
  quantity INT, warehouse_id VARCHAR(10), warehouse_city VARCHAR(80),
  tag_list VARCHAR(500), colour_list VARCHAR(500)
);
```

8.3 Constraint and anomaly tests

```
-- Valid parent rows must be inserted before child rows.
-- This insert must fail: product references an unknown supplier.
INSERT INTO product VALUES ('PX','Test item',10,'CAT1','S999');

-- This insert must fail: negative quantity violates CHECK.
INSERT INTO order_item VALUES ('O101','P01',-2,'W1');

-- This insert must fail: duplicate customer email violates UNIQUE.
INSERT INTO customer VALUES ('C99','Test','asha@mail.test');

-- Update supplier once in normalized design.
UPDATE supplier SET supplier_phone='900009' WHERE supplier_id='S01';

-- The flat table requires updating every row containing S01.
UPDATE order_line_flat SET supplier_phone='900009' WHERE supplier_id='S01';
```

For a reproducible loading experiment, generate one common CSV dataset at each scale (1,000; 10,000; 100,000; 1,000,000 order lines), then load the same source data into flat and normalized schemas. Record rejected rows and test the boundary cases: a product with no tags (valid product row, zero child rows), a product never ordered (valid product row, no order_item row), deleting the only order of a customer (customer remains), and changing a supplier phone (one supplier row changes).

To demonstrate anomalies on the flat table: attempt to add a supplier without an order; delete the only order line carrying a product's only copy; and update only one of several repeated supplier rows to create inconsistent phone values. In the normalized schema, the independent master rows prevent these effects.

TASK 9 – Record Experimental Evidence (3 Marks)

The assignment requires measured storage, row counts, duplicate fact copies, timings, rows touched and constraint violations. These values depend on the MySQL version, hardware, indexes, buffer pool and generated data. They must be obtained by running the implementation; inventing numbers would not be valid experimental evidence.

Order lines	Flat rows / bytes	Normalized rows / bytes	Supplier-name copies in flat	Constraint violations
1,000	Record after run	Record after run	Record after run	Record after run
10,000	Record after run	Record after run	Record after run	Record after run
100,000	Record after run	Record after run	Record after run	Record after run
1,000,000	Record after run	Record after run	Record after run	Record after run

Operation	Flat schema: time / rows touched	Normalized schema: time / rows touched
Update supplier phone	Measure with timer and affected rows	Measure with timer and affected rows
Order history for one customer (5-table join)	Measure elapsed time and EXPLAIN rows	Measure elapsed time and EXPLAIN rows
Load dataset	Record elapsed time and rejected rows	Record elapsed time and rejected rows

```
-- Storage and row-count evidence (run for each schema).
SELECT table_name, table_rows, data_length + index_length AS bytes
FROM information_schema.tables
WHERE table_schema = 'shopnest';

-- Query plan and rows examined.
EXPLAIN ANALYZE
SELECT oh.order_id, oh.order_date, p.product_name, oi.quantity,
```

```
    oi.quantity * p.unit_price AS current_extended_amount,
    w.warehouse_city
FROM order_header oh
JOIN order_item oi ON oi.order_id = oh.order_id
JOIN product p ON p.product_id = oi.product_id
JOIN warehouse w ON w.warehouse_id = oi.warehouse_id
JOIN customer c ON c.customer_id = oh.customer_id
WHERE c.customer_id = 'C01'
ORDER BY oh.order_date DESC;
```

Record exact date/time, MySQL version, CPU/RAM, storage device, index definitions, dataset seed and number of repetitions. Run each query several times, report the median, and distinguish cold-cache from warm-cache results. The sample output cells above are deliberately left for actual observations.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

- Theory predicts high repeated storage in the flat schema: supplier name/phone repeats once per order line for that supplier; product name and current price repeat once per line for that product; customer and address facts repeat once per line in the customer's orders.
- Normalized updates should touch one master row for supplier, product, customer or category facts. The flat schema may touch many order-line rows and has a greater risk of inconsistent updates.
- A flat order-history query may require fewer joins because attributes are in one table. The normalized schema needs joins, but indexes on foreign keys and selective filters reduce lookup cost. With 200,000 queries/day, query plans and caching become important.
- Indexes increase storage and write cost but can reduce rows scanned. Foreign-key checks add insert/update work. Page size and row width affect I/O. Skewed popularity means a small number of products/suppliers may account for many rows and dominate repeated-fact counts.

After Task 9 measurements, compare predicted and observed ratios. If normalized joins are slower, report the measured difference and test composite/covering indexes rather than abandoning integrity. Do not claim a measured winner until the benchmark is run.

TASK 11 – Recommend and Defend the Final Schema (4 Marks)

Recommendation: keep the master and transaction data in BCNF where the dependencies permit it, and keep independent tag/colour facts in 4NF child relations. Keep SUPPLY in the three-relation 5NF form only if the cyclic supplier-category-warehouse rule is confirmed as a real constraint and applications need the pairwise facts independently.

Relation	Recommended form	Reason
CUSTOMER, SUPPLIER, CATEGORY, WAREHOUSE,	BCNF	Each determinant is a candidate key; descriptive

POSTAL		facts are stored once.
PRODUCT	BCNF	ProductID determines its current descriptive and pricing attributes.
ORDER_HEADER, ORDER_ITEM	BCNF	Order and line facts are separated; line key is composite.
ADDRESS	BCNF	AddressID and (street, city) identify an address under the stated rule.
PRODUCT_TAG, PRODUCT_COLOUR	4NF	Independent multivalued facts are stored separately.
SUPPLIER_CATEGORY, CATEGORY_WAREHOUSE, SUPPLIER_WAREHOUSE	5NF, if JD is valid	Avoids storing only the ternary combination and enforces the cyclic rule by joining all three pair facts.

The design preserves customer, product, supplier and order integrity and reduces update anomalies. Its cost is additional joins, indexes and maintenance. At 20,000 new lines per day, normalized inserts are manageable with batch loading and appropriate indexes. As orders grow, partitioning or archival may be considered after measuring workload. The recommendation balances data integrity with query performance; denormalization should be selective and supported by measured need.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

- The changed requirement adds two pressures: customers must see the exact historical price paid, and order-history queries occur 200,000 times per day.
- UnitPrice is no longer determined by ProductID alone when interpreted as the price paid on a particular order. ProductID → UnitPrice can continue to represent the current catalogue price only. The historical paid price depends on the order line (and potentially price/discount/tax rules).
- The high query frequency makes the cost of joining multiple tables important. Historical facts must remain stable even when current product prices change; therefore the historical paid price belongs with the transaction line.

B. Evaluation of the Original Design (2 Marks)

If ORDER_ITEM stores no historical price and the application joins PRODUCT.UnitPrice, an old order will display the latest catalogue price after a weekly price change. That is incorrect because the price may have changed since purchase.

A five-table order-history query repeated 200,000 times per day can create significant join and I/O work. However, normalized joins are not automatically too slow: primary-key/foreign-key indexes, a covering index, caching and a selective customer/order filter can make them efficient.

A separate PRICE_HISTORY(ProductID, ValidFrom, ValidTo, UnitPrice) relation can reconstruct catalogue price at order time, but adds a date-range join and requires unambiguous effective periods. Storing the actual paid price directly in ORDER_ITEM is simpler and preserves the exact transaction fact. A covering index can further accelerate common history queries.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the BCNF design, not discard it. Add UnitPricePaid to ORDER_ITEM. Keep PRODUCT.UnitPrice as the current catalogue price. The paid price is a transaction snapshot and must not be updated when the catalogue price changes.

```
ALTER TABLE order_item
  ADD COLUMN unit_price_paid DECIMAL(10,2) NOT NULL
  CHECK (unit_price_paid >= 0);

-- Example order-history query using the immutable paid price.
SELECT oh.order_id, oh.order_date, p.product_name,
       oi.quantity, oi.unit_price_paid,
       oi.quantity * oi.unit_price_paid AS line_total
FROM order_header oh
JOIN order_item oi ON oi.order_id = oh.order_id
JOIN product p ON p.product_id = oi.product_id
WHERE oh.customer_id = 'C01'
ORDER BY oh.order_date DESC;

-- Candidate covering index for customer order history:
CREATE INDEX idx_order_customer_date
ON order_header(customer_id, order_date, order_id);

-- Supports line retrieval and contains the paid price:
CREATE INDEX idx_item_order_product_price
ON order_item(order_id, product_id, quantity, unit_price_paid);
```

The new dependency is (OrderID, ProductID) → UnitPricePaid, because one order line records one agreed price paid. It does not create an update anomaly: it records a historical transaction fact that is expected to differ across orders and remain unchanged. It is not functionally determined by ProductID alone. Storage increases by one decimal value per order line, but historical queries avoid a price-history range join and return the correct amount. CHECK

prevents negative prices; the composite primary key ensures one paid price per order-product line.

D. Evidence-Based Defence (2 Marks)

The modified design follows from the dependencies and workload:

- Dependency theory: current catalogue price remains ProductID → UnitPrice; actual paid price becomes line-specific, (OrderID, ProductID) → UnitPricePaid.
- Normal forms: ORDER_ITEM remains in BCNF because its determinant is the candidate key. The extra attribute depends on the whole key, not on only OrderID or ProductID.
- Price-change frequency: weekly catalogue changes must not rewrite historical transactions. Storing UnitPricePaid makes the order record stable.
- Query frequency: 200,000 history queries per day justifies indexes aligned with customer/date and order-item access. A covering index may reduce table lookups, at the cost of index storage and write overhead.
- Dataset size: one extra DECIMAL field per line adds predictable storage, while avoiding repeated historical price reconstruction. At one million lines, the additional column is stored once per line.
- Integrity-performance trade-off: normalized master data avoids supplier/customer/product update anomalies; a small transaction-level snapshot and targeted indexes improve correctness and history-query speed.

Final decision: retain the normalized BCNF/4NF/conditional-5NF structure and modify ORDER_ITEM by adding UnitPricePaid, then benchmark the indexed history query. This is the best fit for the changed requirement because it preserves historical truth without broadly denormalizing the database.

Conclusion

The original flat ORDER_LINE relation combines order, customer, product, supplier, category, warehouse, address and independent multivalued facts. Its functional and multivalued dependencies explain the resulting redundancy and anomalies. Decomposition through 2NF, 3NF, BCNF and 4NF separates facts according to their determinants; the conditional 5NF decomposition handles the cyclic SUPPLY join dependency. Under the revised requirement, the design should be modified by storing the exact price paid on each order line and adding workload-appropriate indexes. Actual performance claims must be completed using the reproducible experiments in Tasks 8–10.

References / implementation notes

- Supplied source: DATABASE MANAGEMENT SYSTEMS (CSA0501), Assignment 3 – CO3, ShopNest online retail order-management scenario, 7 pages.
- SQL syntax is written for MySQL 8+. CHECK constraints are enforced in current MySQL 8 releases; verify the exact server version before execution.

- The sample dataset is illustrative and constructed for this assignment. It is not a production ShopNest export or a measured benchmark.